

✓ Módulo 3: Entrada, Saída e Formatação de Texto

Certifique-se de que você está dentro da pasta da aula antes de executar o comando abaixo. O comando criará automaticamente as subpastas `cpp/src`, `cpp/bin` e `cpp/lib` dentro dela. Caso tenha executado anteriormente, não é necessário reexecutar.

```
!mkdir -p cpp/src cpp/bin cpp/lib
```

253.32s - pydevd: Sending message related to process being replaced timed-out after 5 seconds

A estrutura gerada será a seguinte.

```
NOME_DA_AULA/
├── cpp/
│   ├── src/
│   ├── bin/
│   └── lib/
```

Pasta	Conteúdo	Extensões
<code>src</code>	Arquivos fonte com a lógica do programa	<code>.cpp</code>
<code>bin</code>	Executáveis gerados pelo compilador	sem extensão
<code>lib</code>	Cabeçalhos e bibliotecas externas	<code>.h</code> <code>.hpp</code>

Nota sobre os comandos do Jupyter Notebook

Você vai notar que os exemplos desta aula utilizam alguns comandos especiais, como `%%writefile`, `!g++` e `!./...`. É importante entender que **esses prefixos não fazem parte do C++**. O `%%writefile` e o `!` são apenas uma "ponte" entre o Jupyter e o terminal do sistema. **O fluxo real é o mesmo que você vê nos vídeos**: escrever o código, compilar com `g++` e executar o binário. — eles são recursos do próprio Jupyter Notebook que nos permitem escrever e executar código C++ diretamente por aqui.

- `%%writefile cpp/src/02_exec_02.cpp` salva o conteúdo da célula como um arquivo `.cpp` no disco.
- `!g++ cpp/src/02_exec_02.cpp -o cpp/bin/02_exec_02` compila o arquivo usando o `g++`, exatamente como você faria no terminal.
- `!./cpp/bin/02_exec_02` executa o binário gerado.

Disponibilizamos os exemplos dessa forma para que você consiga acompanhar e rodar tudo diretamente na aula, sem precisar sair do notebook, de modo online e interativo. Mas recomendamos que, sempre que possível, **pratique também pelo terminal**, como demonstrado nos vídeos — é assim que você vai trabalhar no dia a dia com C++.

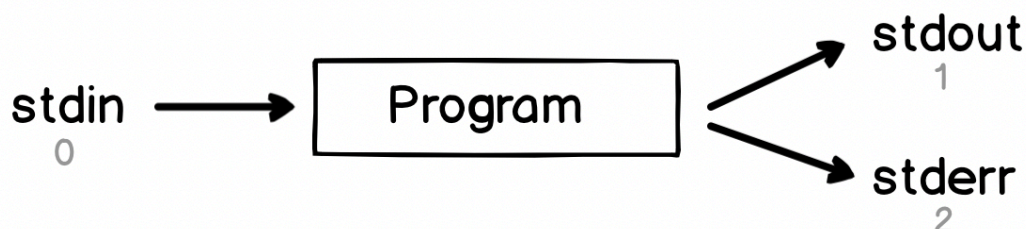
✓ 3.1. Canais Básicos (`<iostream>`)

Quando você executa um programa no terminal, o sistema operacional cria automaticamente **três canais de comunicação** entre o programa e o mundo exterior. Em C++, esses canais são chamados de *streams* (fluxos), mas na prática, pense neles como **tubos invisíveis** conectando o seu programa ao teclado e à tela.

Esses três canais são conceitos do sistema operacional (Linux, Windows, macOS), não exclusivos de C++. Qualquer linguagem de programação os utiliza:

Canal	Nome técnico	Direção	Onde vai/vem
<code>stdin</code>	Standard Input	Entrada → Programa	Teclado (por padrão)
<code>stdout</code>	Standard Output	Programa → Saída	Terminal/Tela (por padrão)
<code>stderr</code>	Standard Error	Programa → Saída	Terminal/Tela (por padrão)

Por que existem dois canais de saída? Imagine que você redireciona a saída do programa para um arquivo (`./programa > resultado.txt`). Nesse caso, tudo que for para `stdout` vai para o arquivo. Mas se houver um erro, você quer vê-lo na tela, não perdido dentro do arquivo. Para isso existe o `stderr`: um canal separado só para erros e diagnósticos.



Esses canais existem independentemente da linguagem — um programa em Python, Java ou Rust acessa os mesmos `stdin`, `stdout` e `stderr`. O que muda é **como** cada linguagem permite que você interaja com eles. Em C++, a biblioteca `<iostream>` cria objetos prontos que funcionam como "portas de acesso" para cada canal:

Canal do SO	Objeto C++	Operador	O que faz
<code>stdin</code>	<code>cin</code>	<code>>></code>	Extrai (puxa) dados do teclado
<code>stdout</code>	<code>cout</code>	<code><<</code>	Insere (empurra) dados para a tela
<code>stderr</code>	<code>cerr</code>	<code><<</code>	Envia erros direto, sem buffer
<code>stderr</code>	<code>clog</code>	<code><<</code>	Envia logs com buffer

Para ter acesso a esses objetos, inclua o cabeçalho no início do programa:

```
# include <iostream>
using namespace std;
```

3.1.1. `cout` — Saída Padrão (stdout)

O objeto `cout` envia dados para a **saída padrão** (`stdout`), que normalmente é o terminal. Ele utiliza o **operador de inserção** `<<`, que "empurra" dados para o canal de saída. Você pode encadear múltiplos `<<` na mesma linha para enviar vários dados de uma vez. Analise o exemplo abaixo:

```
%%writefile cpp/src/03_exec_01.cpp
#include <iostream>
using namespace std;

int main() {

    // Enviar texto simples para o terminal
    cout << "Hello, World!" << endl;

    // Encadeando múltiplos dados com <<
    int x = 42;
    cout << "O valor de x é: " << x << endl;

    // Múltiplas inserções na mesma linha com diferentes tipos
    int versao = 20;
    double nota = 9.5;
    char conceito = 'A';
    cout << "C" << versao << " | Nota: " << nota << " | Conceito: " << conceito << endl;

    return 0;
}
```

Para compilar o programa acima, rode o comando abaixo:

```
!g++ cpp/src/03_exec_01.cpp -o cpp/bin/03_exec_01
```

Para executar o programa, você tem duas opções: rode o binário diretamente pelo terminal (nativo ou integrado do VSCode) navegando até a pasta da aula e executando `./cpp/bin/03_exec_01`, ou utilize a célula abaixo no próprio Jupyter Notebook para facilitar.

```
!./cpp/bin/03_exec_01
```

✓ 3.1.2. `cin` — Entrada Padrão (stdin)

O objeto `cin` lê dados da **entrada padrão** (`stdin`), que normalmente é o teclado. Ele utiliza o **operador de extração** `>>`, que "puxa" dados do canal de entrada e os armazena em variáveis.

```
%%writefile cpp/src/03_exec_02.cpp
#include <iostream>
using namespace std;

int main() {

    int idade;
    cout << "Digite sua idade: ";
    cin >> idade;
    cout << "Você tem " << idade << " anos." << endl;

    return 0;
}
```

Para compilar o programa acima, rode o comando abaixo:

```
!g++ cpp/src/03_exec_02.cpp -o cpp/bin/03_exec_02
```

Para executar o programa, você tem duas opções: rode o binário diretamente pelo terminal (nativo ou integrado do VSCode) navegando até a pasta da aula e executando `./cpp/bin/03_exec_02`, ou utilize a célula abaixo no próprio Jupyter Notebook para facilitar.

```
!./cpp/bin/03_exec_02
```

Lendo múltiplos valores: Você pode usar `cin >>` várias vezes para ler dados de tipos diferentes em sequência. O operador `>>` também pode ser encadeado:

```
%writefile cpp/src/03_exec_03.cpp
#include <iostream>
using namespace std;

int main() {

    int idade;
    double altura;
    char inicial;

    cout << "Digite sua idade, altura e inicial do nome:" << endl;
    cout << "(separados por espaço, ex: 25 1.75 J)" << endl;
    cin >> idade >> altura >> inicial;

    cout << "Idade: " << idade << " anos" << endl;
    cout << "Altura: " << altura << " m" << endl;
    cout << "Inicial: " << inicial << endl;

    return 0;
}
```

Para compilar o programa acima, rode o comando abaixo:

```
!g++ cpp/src/03_exec_03.cpp -o cpp/bin/03_exec_03
```

Para executar o programa, você tem duas opções: rode o binário diretamente pelo terminal (nativo ou integrado do VSCode) navegando até a pasta da aula e executando `./cpp/bin/03_exec_03`, ou utilize a célula abaixo no próprio Jupyter Notebook para facilitar.

```
!./cpp/bin/03_exec_03
```

Comportamento importante: `cin >>` ignora espaços em branco (espaços, tabs, quebras de linha) por padrão. Ele lê valor por valor, pulando os espaços entre eles. Para ler textos com espaços (como um nome completo), existem outras técnicas que veremos na aula dedicada a **strings**.

3.1.3. `cerr` — Saída de Erro (stderr, sem buffer)

O `cerr` envia dados para o canal de erro padrão (`stderr`). A diferença crucial em relação ao `cout` é que ele **não possui buffer** (*unbuffered*): cada mensagem é exibida **imediatamente**, sem esperar acumular dados.

Por que isso importa? Quando um programa trava, o conteúdo que ainda está no buffer de `cout` pode se perder — ele não teve tempo de ser enviado à tela. Mensagens de `cerr` nunca se perdem, pois saem instantaneamente.

```
%writefile cpp/src/03_exec_04.cpp
#include <iostream>
using namespace std;

int main() {

    cout << "Início do programa." << endl;
    cerr << "ERRO: arquivo não encontrado!" << endl;
    cout << "Fim do programa." << endl;

    return 0;
}
```

Para compilar o programa acima, rode o comando abaixo:

```
!g++ cpp/src/03_exec_04.cpp -o cpp/bin/03_exec_04
```

Para executar o programa, você tem duas opções: rode o binário diretamente pelo terminal (nativo ou integrado do VSCode) navegando até a pasta da aula e executando `./cpp/bin/03_exec_04`, ou utilize a célula abaixo no próprio Jupyter Notebook para facilitar.

```
!./cpp/bin/03_exec_04
```

Ao executar, repare que tanto as mensagens de `cout` quanto a de `cerr` aparecem juntas no terminal. Visualmente parece que usam o mesmo caminho, mas na verdade são dois canais completamente independentes. O sistema operacional separa a saída normal (`stdout`) da saída de erros (`stderr`) de propósito. A ideia é simples. Imagine um programa que processa milhares de registros e grava os resultados em um arquivo. Se durante esse processamento ocorrer um erro, você não quer que a mensagem de erro se misture com os dados processados. Você quer ver o erro imediatamente na tela, mesmo que toda a saída normal esteja indo para o arquivo. É exatamente isso que o `stderr` permite.

Além disso, o `cerr` não possui buffer. Isso significa que cada mensagem enviada por ele aparece instantaneamente no destino. Já o `cout` acumula dados em um buffer antes de enviá-los. Se o programa travar no meio da execução, o conteúdo que ainda estava acumulado no buffer de `cout` pode se perder, pois não teve tempo de ser enviado à tela. Mensagens de `cerr` nunca se perdem, pois já foram exibidas no momento em que o programa as enviou. Essa garantia torna o `cerr` a escolha correta para qualquer mensagem que o programador precisa ver aconteça o que acontecer.

Podemos provar essa separação de canais com **redirecionamento**.

Experimento prático com redirecionamento. Execute o mesmo programa redirecionando a saída padrão para um arquivo.

```
./cpp/bin/03_exec_04 > saida.txt
```

O operador `>` captura apenas o canal `stdout`, identificado pelo número `1` no sistema operacional, e grava seu conteúdo no arquivo. Como o `stderr`, identificado pelo número `2`, não foi redirecionado, apenas a mensagem de `cerr` aparece no terminal. Para conferir o que foi capturado no arquivo.

```
cat saida.txt
```

Podemos fazer o inverso. Redirecionar apenas o `stderr` usando o operador `2>`.

```
./cpp/bin/03_exec_04 2> erros.txt
```

Agora as mensagens de `cout` aparecem no terminal e o erro fica no arquivo. Confira com.

```
cat erros.txt
```

Também é possível separar cada canal para um arquivo diferente.

```
./cpp/bin/03_exec_04 > saida.txt 2> erros.txt
```

Nada aparece no terminal, pois cada canal foi direcionado para seu próprio arquivo. Confira com `cat saida.txt` e `cat erros.txt`.

Por fim, se quiser juntar tudo no mesmo arquivo.

```
./cpp/bin/03_exec_04 &> tudo.txt
```

O operador `&>` combina `stdout` e `stderr` no mesmo destino. Na prática, programas bem escritos usam `cout` para dados e resultados, `cerr` para erros críticos e `clog` para mensagens de diagnóstico. Essa separação permite que quem executa o programa decida o que fazer com cada tipo de informação, redirecionando cada canal conforme a necessidade. Esse conceito não é exclusivo do C++. Qualquer linguagem de programação acessa os mesmos canais do sistema operacional.

3.1.4. `clog` — Log de Diagnóstico (`stderr`, com buffer)

O `clog` também envia para `stderr`, porém **possui buffer** (*buffered*). Na prática, isso significa que o sistema acumula várias mensagens antes de enviá-las de uma vez ao destino, o que é mais eficiente quando há muitas mensagens em sequência. Como vai

para `stderr`, as mensagens de `clog` podem ser separadas da saída normal do programa por redirecionamento (assim como o `cerr`).

Quando usar `clog` em vez de `cerr`? Use `cerr` para erros que precisam aparecer imediatamente (um crash, uma falha crítica). Use `clog` para mensagens informativas de acompanhamento (logs de diagnóstico, progresso), onde a eficiência importa mais que a imediatez.

```
%writefile cpp/src/03_exec_05.cpp
#include <iostream>
using namespace std;

int main() {

    cout << "Resultado: operação concluída com sucesso." << endl;
    cout << "Resultado: 42 registros processados." << endl;

    clog << "LOG: conexão estabelecida com o servidor." << endl;
    clog << "LOG: processando dados..." << endl;
    clog << "LOG: operação concluída." << endl;

    cerr << "ERRO: falha ao gravar registro 17." << endl;

    return 0;
}
```

Para compilar o programa acima, rode o comando abaixo:

```
!g++ cpp/src/03_exec_05.cpp -o cpp/bin/03_exec_05
```

Para executar o programa, você tem duas opções: rode o binário diretamente pelo terminal (nativo ou integrado do VSCode) navegando até a pasta da aula e executando `./cpp/bin/03_exec_05`, ou utilize a célula abaixo no próprio Jupyter Notebook para facilitar.

```
!./cpp/bin/03_exec_05
```

Ao executar, todas as mensagens aparecem juntas no terminal. Como mencionado acima, o `clog` envia tudo para `stderr`, assim como o `cerr`. Isso significa que podemos usar o mesmo redirecionamento que aprendemos na seção anterior para separar cada tipo de informação. Experimente os comandos abaixo no terminal.

Primeiro, execute o programa normalmente para ver tudo junto no terminal.

```
./cpp/bin/03_exec_05
```

Agora, use o operador `>` para redirecionar o `stdout` para um arquivo. O `>` sozinho equivale a `1>`, pois o número `1` é o identificador que o sistema operacional atribui ao canal `stdout`. Como não redirecionamos o canal `2` (`stderr`), as mensagens de `cerr` e `clog` continuam aparecendo no terminal.

```
./cpp/bin/03_exec_05 > resultados.txt
```

Faça o inverso. Use `2>` para redirecionar apenas o canal `2`, que é o `stderr`. Dessa vez, as mensagens de `cerr` e `clog` vão para o arquivo, e as de `cout` (canal `1`) continuam no terminal.

```
./cpp/bin/03_exec_05 2> logs.txt
```

Também é possível combinar os dois operadores no mesmo comando, enviando cada canal para um arquivo diferente. Nesse caso, nada aparece no terminal, pois tanto o canal `1` quanto o `2` foram redirecionados.

```
./cpp/bin/03_exec_05 > resultados.txt 2> logs.txt
```

Para conferir o conteúdo capturado em cada arquivo.

```
cat resultados.txt
cat logs.txt
```

Na prática, esse é um padrão muito comum. O programa exibe seus resultados via `cout` (que vai para `stdout`) e registra o que está fazendo via `clog` e `cerr` (que vão para `stderr`). Quem executa o programa pode então decidir gravar os logs em arquivo sem afetar a saída principal. Um ponto importante. O `cerr` e o `clog` vão ambos para `stderr`, então não é possível separá-los em

arquivos diferentes usando apenas redirecionamento do terminal. Do ponto de vista do sistema operacional, são o mesmo canal. A diferença entre eles é apenas interna ao C++, no comportamento do buffer.

Resumo dos operadores de redirecionamento

Ao longo das seções 3.1.3, 3.1.4 e 3.1.5, utilizamos diversos operadores de redirecionamento do terminal. A tabela abaixo consolida todos eles para consulta rápida.

Operador	O que faz	Identificador
>	Redireciona <code>stdout</code> para arquivo	1
2>	Redireciona <code>stderr</code> para arquivo	2
&>	Redireciona ambos para o mesmo arquivo	1 + 2
>>	Igual ao >, mas adiciona ao final	1
2>>	Igual ao 2>, mas adiciona ao final	2

Resumo dos canais de I/O

Antes de avançar para formatação de saída, vale consolidar os quatro objetos que vimos até aqui e o papel de cada um.

Canal C++	Destino	Buffer	Uso típico
<code>cout</code>	<code>stdout</code>	Sim	Saída normal do programa
<code>cin</code>	<code>stdin</code>	Sim	Leitura de dados do usuário
<code>cerr</code>	<code>stderr</code>	Não	Mensagens de erro (imediatas)
<code>clog</code>	<code>stderr</code>	Sim	Mensagens de log e diagnóstico

3.2. Formatação de Saída (<iomanip>)

A biblioteca `<iomanip>` fornece **manipuladores** que controlam a aparência dos dados impressos com `cout`. Útil para montar tabelas alinhadas, controlar casas decimais e formatar valores booleanos.

```
# include <iomanip>
```

3.2.1. `setw(n)` — Largura de Campo

O manipulador `setw(n)` define a **largura mínima** do próximo campo a ser impresso. Se o dado ocupar menos de `n` caracteres, o espaço restante é preenchido com espaços em branco (ou com o caractere definido por `setfill`, que veremos adiante). Se o dado for maior que `n`, ele é impresso normalmente sem corte.

Atenção: `setw` afeta **apenas** a próxima impressão. Você precisa aplicá-lo antes de cada valor. Esse é o único manipulador que não é persistente.

```
%writefile cpp/src/03_exec_06.cpp
#include <iostream>
using namespace std;

int main() {

    cout << "=== Sem formatação ===" << endl;
    cout << "Ana" << " " << 25 << endl;
    cout << "João" << " " << 30 << endl;
    cout << "Maria" << " " << 8 << endl;

    cout << endl;

    cout << "=== Com setw ===" << endl;
    cout << setw(10) << "Nome" << setw(10) << "Idade" << endl;
    cout << setw(10) << "Ana" << setw(10) << 25 << endl;
    cout << setw(11) << "João" << setw(10) << 30 << endl;
    cout << setw(10) << "Maria" << setw(10) << 8 << endl;

    return 0;
}
```

Para compilar o programa acima, rode o comando abaixo:

```
!g++ cpp/src/03_exec_06.cpp -o cpp/bin/03_exec_06
```

Para executar o programa, você tem duas opções: rode o binário diretamente pelo terminal (nativo ou integrado do VSCode) navegando até a pasta da aula e executando `./cpp/bin/03_exec_06`, ou utilize a célula abaixo no próprio Jupyter Notebook para facilitar.

```
!./cpp/bin/03_exec_06
```

Compare as duas saídas. Sem `setw`, os dados ficam grudados e desalinhados. Com `setw(10)`, cada campo ocupa no mínimo 10 caracteres, criando colunas uniformes. Repare que, por padrão, o conteúdo fica alinhado à direita dentro do campo.

3.2.2. `left`, `right` e `internal` — Alinhamento

Como vimos acima, o alinhamento padrão dentro de um campo `setw` é à **direita** (`right`). Podemos mudar esse comportamento com os manipuladores `left`, `right` e `internal`.

Diferente do `setw`, esses manipuladores são **persistentes**, ou seja, uma vez aplicados, permanecem ativos para todas as impressões seguintes até que você os altere.

O manipulador `internal` é específico para números com sinal. Ele coloca o sinal na extremidade esquerda do campo e o valor na extremidade direita, preenchendo o espaço do meio.

```
%%writefile cpp/src/03_exec_07.cpp
#include <iostream>
using namespace std;

int main() {

    cout << "=== Alinhamento à direita (padrão) ===" << endl;
    cout << right;
    cout << setw(10) << "Nome" << setw(10) << "Idade" << endl;
    cout << setw(10) << "Ana" << setw(10) << 25 << endl;
    cout << setw(10) << "João" << setw(10) << 30 << endl;

    cout << endl;

    cout << "=== Alinhamento à esquerda ===" << endl;
    cout << left;
    cout << setw(10) << "Nome" << setw(10) << "Idade" << endl;
    cout << setw(10) << "Ana" << setw(10) << 25 << endl;
    cout << setw(10) << "João" << setw(10) << 30 << endl;

    cout << endl;

    cout << "=== Alinhamento interno (sinal separado do valor) ===" << endl;
    cout << internal;
    cout << setw(10) << -42.5 << endl;
    cout << setw(10) << -7.0 << endl;

    return 0;
}
```

Para compilar o programa acima, rode o comando abaixo:

```
!g++ cpp/src/03_exec_07.cpp -o cpp/bin/03_exec_07
```

Para executar o programa, você tem duas opções: rode o binário diretamente pelo terminal (nativo ou integrado do VSCode) navegando até a pasta da aula e executando `./cpp/bin/03_exec_07`, ou utilize a célula abaixo no próprio Jupyter Notebook para facilitar.

```
!./cpp/bin/03_exec_07
```

Observe como o mesmo dado muda de posição dentro do campo conforme o alinhamento escolhido. No modo `internal`, repare que o sinal de menos fica colado à esquerda e o número fica colado à direita, com os espaços no meio.

3.2.3. `setfill(c)` — Caractere de Preenchimento

Por padrão, o espaço vazio dentro de um campo `setw` é preenchido com espaços em branco. O manipulador `setfill(c)` permite trocar esse caractere por qualquer outro. Assim como `left` e `right`, ele é **persistente**.

```
%%writefile cpp/src/03_exec_08.cpp
#include <iostream>
using namespace std;

int main() {

    cout << "=== Preenchimento com pontos ===" << endl;
    cout << setfill('.') << left;
```

```

cout << setw(20) << "Café"      << " R$ 5.90" << endl;
cout << setw(20) << "Bolo"      << " R$ 12.50" << endl;
cout << setw(20) << "Suco"      << " R$ 7.00" << endl;

cout << endl;

cout << "=== Preenchimento com traços ===" << endl;
cout << setfill('-');
cout << setw(20) << "Subtotal"   << " R$ 25.40" << endl;

return 0;
}

```

Para compilar o programa acima, rode o comando abaixo:

```
!g++ cpp/src/03_exec_08.cpp -o cpp/bin/03_exec_08
```

Para executar o programa, você tem duas opções: rode o binário diretamente pelo terminal (nativo ou integrado do VSCode) navegando até a pasta da aula e executando [./cpp/bin/03_exec_08](#), ou utilize a célula abaixo no próprio Jupyter Notebook para facilitar.

```
!./cpp/bin/03_exec_08
```

Esse padrão de preenchimento com pontos é muito usado em menus, sumários e recibos para guiar o olho do leitor do nome até o valor.

3.2.4. `fixed` e `setprecision(n)` — Casas Decimais

Por padrão, `cout` imprime números de ponto flutuante com até 6 dígitos significativos e decide sozinho quando usar notação científica. Em muitas situações, como exibir valores monetários ou resultados de cálculos, precisamos controlar exatamente quantas casas decimais aparecem. Para isso, combinamos dois manipuladores.

O `fixed` força a exibição em notação fixa (sem notação científica). O `setprecision(n)` define quantas casas decimais serão mostradas. Ambos são **persistentes**. Para voltar ao comportamento padrão, é necessário desativar os dois.

```
cout.unsetf(ios::scientific | ios::fixed);
```

```

%%writefile cpp/src/03_exec_09.cpp
#include <iostream>
using namespace std;

int main() {

    double pi = 3.14159265358979;

    cout << "=== Controle de casas decimais ===" << endl;
    cout << "Padrão:      " << pi << endl;

    cout << fixed << setprecision(2);
    cout << "2 casas:    " << pi << endl;

    cout << setprecision(6);
    cout << "6 casas:      " << pi << endl;

    cout << endl;

    cout << "=== Notação científica ===" << endl;
    cout << scientific << setprecision(3);
    double grande = 6022000000000000000000.0;
    double pequeno = 0.0000000001;
    cout << "Avogadro:      " << grande << endl;
    cout << "Pequeno:       " << pequeno << endl;

    cout << endl;

    return 0;
}

```

Para compilar o programa acima, rode o comando abaixo:

```
!g++ cpp/src/03_exec_09.cpp -o cpp/bin/03_exec_09
```


Para executar o programa, você tem duas opções: rode o binário diretamente pelo terminal (nativo ou integrado do VSCode) navegando até a pasta da aula e executando `./cpp/bin/03_exec_09`, ou utilize a célula abaixo no próprio Jupyter Notebook para facilitar.

```
!./cpp/bin/03_exec_09
```

Repare que `setprecision` funciona de forma diferente dependendo do modo ativo. Com `fixed`, ele define o número de casas decimais depois da vírgula. Com `scientific`, ele define o número de casas decimais na mantissa. Sem nenhum dos dois (modo padrão), ele define o total de dígitos significativos.

3.2.5. `boolalpha` — Booleanos como Texto

Por padrão, o C++ imprime valores `bool` como números. O valor `true` aparece como `1` e `false` como `0`. Em muitas situações, isso dificulta a leitura da saída. O manipulador `boolalpha` faz com que o `cout` exiba as palavras `true` e `false` em vez dos números. Para reverter ao comportamento original, use `noboolalpha`.

```
%%writefile cpp/src/03_exec_10.cpp
#include <iostream>
using namespace std;

int main() {

    bool ativo = true;
    bool finalizado = false;

    cout << "=== Sem boolalpha (padrão) ===" << endl;
    cout << "ativo:      " << ativo << endl;
    cout << "finalizado: " << finalizado << endl;

    cout << endl;

    cout << "=== Com boolalpha ===" << endl;
    cout << boolalpha;
    cout << "ativo:      " << ativo << endl;
    cout << "finalizado: " << finalizado << endl;

    cout << endl;

    cout << "=== Desativando com noboolalpha ===" << endl;
    cout << noboolalpha;
    cout << "ativo:      " << ativo << endl;
    cout << "finalizado: " << finalizado << endl;

    return 0;
}
```

Para compilar o programa acima, rode o comando abaixo:

```
!g++ cpp/src/03_exec_10.cpp -o cpp/bin/03_exec_10
```

Para executar o programa, você tem duas opções: rode o binário diretamente pelo terminal (nativo ou integrado do VSCode) navegando até a pasta da aula e executando `./cpp/bin/03_exec_10`, ou utilize a célula abaixo no próprio Jupyter Notebook para facilitar.

```
!./cpp/bin/03_exec_10
```

3.2.6. Outros Manipuladores Úteis

A tabela abaixo lista manipuladores adicionais que você pode explorar conforme a necessidade. A maioria está disponível em `<iostream>` ou `<iomanip>`. Não serão todos cobertos em aula, mas vale conhecer e experimentar por conta própria.

Manipulador	Cabeçalho	Descrição
<code>showpos</code>	<code><iostream></code>	Exibe o sinal <code>+</code> em números positivos
<code>noshowpos</code>	<code><iostream></code>	Remove o sinal <code>+</code> (padrão)
<code>uppercase</code>	<code><iostream></code>	Letras maiúsculas em hex e notação científica (<code>1.5E+02</code>)
<code>nouppercase</code>	<code><iostream></code>	Letras minúsculas (padrão)
<code>showpoint</code>	<code><iostream></code>	Sempre mostra o ponto decimal e zeros finais
<code>noshowpoint</code>	<code><iostream></code>	Oculto zeros finais desnecessários (padrão)
<code>dec</code>	<code><iostream></code>	Imprime inteiros em base decimal (padrão)

Manipulador	Cabeçalho	Descrição
<code>hex</code>	<code><iostream></code>	Imprime inteiros em base hexadecimal
<code>oct</code>	<code><iostream></code>	Imprime inteiros em base octal
<code>setbase(n)</code>	<code><iomanip></code>	Define a base numérica (8, 10 ou 16)
<code>showbase</code>	<code><iostream></code>	Mostra prefixo da base (<code>0x</code> para hex, <code>0</code> para octal)

3.3. Exemplo Integrado

O programa abaixo reúne os principais conceitos vistos ao longo do módulo em um único fluxo. Ele lê dados do usuário com `cin`, exibe resultados formatados com `cout` e `<iomanip>`, registra informações de diagnóstico com `clog` e reporta um resultado de validação com `cerr`. Ao executar, experimente redirecionar os canais para observar a separação entre `stdout` e `stderr`.

```
%writefile cpp/src/03_exec_11.cpp
#include <iostream>
#include <iomanip>
using namespace std;

int main() {

    // --- Leitura com cin ---
    char inicial;
    int idade;
    double peso;
    double altura;

    cout << "Inicial do nome: ";
    cin >> inicial;

    cout << "Idade: ";
    cin >> idade;

    cout << "Peso (kg): ";
    cin >> peso;

    cout << "Altura (m): ";
    cin >> altura;

    // --- Log de diagnóstico via clog (vai para stderr, com buffer) ---
    clog << "LOG: dados recebidos com sucesso." << endl;

    // --- Cálculos ---
    double imc = peso / (altura * altura);
    bool faixa_saudavel = (imc >= 18.5 && imc < 25.0);

    clog << "LOG: IMC calculado." << endl;

    // --- Saída formatada com cout (vai para stdout) ---
    cout << endl;

    int col = 18;

    cout << "=====" << endl;
    cout << "          Ficha de Saúde" << endl;
    cout << "=====" << endl;

    cout << left;
    cout << setw(col) << "Paciente:" << inicial << "." << endl;
    cout << setw(col) << "Idade:" << idade << " anos" << endl;
    cout << setw(col) << "Peso:" << fixed << setprecision(1) << peso << " kg" << endl;
    cout << setw(col) << "Altura:" << fixed << setprecision(2) << altura << " m" << endl;

    cout << setfill('.');
    cout << setw(col) << "IMC" << " " << fixed << setprecision(1) << imc << endl;
    cout << setfill(' ');

    cout << setw(col) << "Faixa OK?" << boolalpha << faixa_saudavel << endl;

    cout << "=====" << endl;

    // --- Resultado de validação via cerr (vai para stderr, sem buffer) ---
    cerr << "AVISO: consulte um profissional para avaliacao completa." << endl;

    return 0;
}
```

Saída esperada (para entrada: `Maria Santos`, `28`, `4500.50`):

```
===== Ficha Cadastral =====
Nome:      Maria Santos
Idade:     28 anos
Salário:   R$ 4500.50
LOG: cadastro processado com sucesso.
```

Para compilar o programa acima, rode o comando abaixo:

```
!g++ cpp/src/03_exec_11.cpp -o cpp/bin/03_exec_11
```

Para executar o programa, você tem duas opções: rode o binário diretamente pelo terminal (nativo ou integrado do VSCode) navegando até a pasta da aula e executando `./cpp/bin/03_exec_11`, ou utilize a célula abaixo no próprio Jupyter Notebook para facilitar.

```
!./cpp/bin/03_exec_11
```

Saída esperada (para entrada `M`, `28`, `70.5`, `1.75`).

```
=====
          Ficha de Saúde
=====
Paciente:      M.
Idade:         28 anos
Peso:         70.5 kg
Altura:        1.75 m
IMC..... 23.0
Faixa OK?      true
=====
AVISO: consulte um profissional para avaliacao completa.
```

Observe que a mensagem de `cerr` e as de `clog` vão para `stderr`. Para comprovar, redirecione apenas o `stdout` e veja que o aviso e os logs continuam aparecendo no terminal.

```
./cpp/bin/03_exec_11 > ficha.txt
cat ficha.txt
```

O arquivo `ficha.txt` conterá apenas a ficha formatada, enquanto o aviso e os logs permaneceram visíveis no terminal. Esse exemplo demonstra na prática por que a separação entre `stdout` e `stderr` é útil.

Resumo do Módulo

Neste módulo vimos que C++ organiza toda a comunicação de I/O em **streams**. Os quatro objetos principais (`cout`, `cin`, `cerr`, `clog`) cobrem os cenários fundamentais de saída, entrada, erros e logs. A formatação com `<iomanip>` permite construir saídas organizadas e legíveis, desde tabelas alinhadas até valores numéricos com precisão controlada. A tabela da seção 3.2.6 lista manipuladores adicionais que vale explorar por conta própria para ampliar o repertório.